

Extra 5 · Advanced SQL Interview Problems

What this is. The "hard tier" that follows Chapter 16: the problems that separate a good SQL candidate from an average one — **gaps and islands, running and moving averages, month-over-month growth, pivot/unpivot, relational division, de-duplication, top-N per group with ties, median** and more. Each problem has the question, the **idea** (the part interviewers actually want to hear), the query and the **real result** on the sample data.

How to use it. Cover the solution, write your own query, compare. Every query was executed on the sample data, so you can trust the outputs. Tables used are the company tables from Chapter 00 (`emp` , `dept`) plus the small tables shown where they first appear.

#	Problem	Technique
1–2	Find and delete duplicates	<code>GROUP BY ... HAVING , MIN(id) , ROW_NUMBER</code>
3–4	N-th highest (per group, and distinct)	<code>DENSE_RANK</code> , correlated <code>COUNT</code>
5	Median	<code>MEDIAN</code> , portable <code>ROW_NUMBER</code> method
6–8	Consecutive-day streaks, missing numbers	gaps and islands, <code>LEAD</code>
9–11	Running total, moving average, month-over-month growth	window frames, <code>LAG</code>
12–13	Pivot and unpivot	conditional aggregation, <code>UNION ALL</code>
14	Cumulative percentage (Pareto)	<code>SUM()</code> <code>OVER</code>
15	Top-N per group with ties	<code>DENSE_RANK</code>
16	"Bought all products"	relational division
17	Swap values with one <code>UPDATE</code>	<code>CASE</code> in <code>UPDATE</code>
18–20	Self-join puzzles	self join, <code>GROUP BY</code>
21	Rows → comma-separated list	<code>LISTAGG</code>
22	First / last value in a group	<code>FIRST_VALUE</code> , frame trap
23–25	Recent rows, compare with previous row, salary bands	<code>FETCH</code> , <code>LAG</code> , <code>NTILE</code>
26–28	Groups where <i>all</i> rows qualify, cross-department, cumulative headcount	<code>HAVING MIN</code> , NULL trap, nested aggregate

Part A — Duplicates

1. Find duplicate rows

Table `contacts` — the same person was entered several times with the same email.

contacts

id	name	email
1	Asha	asha@x.com
2	Bala	bala@x.com
3	Asha K	asha@x.com
4	Chitra	chi@x.com
5	Bala S	bala@x.com
6	Asha K.	asha@x.com

Idea: group by the column that defines "duplicate" and keep groups with more than one row.

```
SELECT email, COUNT(*) AS times
FROM   contacts
GROUP  BY email
HAVING COUNT(*) > 1
ORDER  BY email;
```

Result

email	times
asha@x.com	3
bala@x.com	2

2. Delete the duplicates but keep one row each

Idea: keep the row with the **smallest id** per email, delete every other row.

```
DELETE FROM contacts
WHERE id NOT IN (SELECT MIN(id) FROM contacts GROUP BY email);
```

Table after

id	name	email
1	Asha	asha@x.com
2	Bala	bala@x.com
4	Chitra	chi@x.com

Two more ways, useful when a table has **no unique id**:

```
-- Oracle only: every row has a hidden ROWID
DELETE FROM contacts c
WHERE c.ROWID > (SELECT MIN(c2.ROWID) FROM contacts c2 WHERE c2.email = c.email);

-- Portable: number the rows inside each group, delete number 2, 3, ...
DELETE FROM contacts
WHERE id IN (SELECT id
             FROM (SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
                  FROM contacts)
             WHERE rn > 1);
```

Always run the SELECT version first (SELECT * ... WHERE id NOT IN ...) to check what would be deleted. To prevent the problem, add a UNIQUE (email) constraint.

Part B — N-th highest and the median

3. Second highest salary in each department

Table `staff(id, dept_id, salary, hire_date)` — deliberately contains ties.

staff

id	dept_id	salary	hire_date
1	10	10000	2020-01-01
2	10	9000	2020-06-01
3	10	9000	2021-01-01
4	10	8000	2022-01-01
5	20	7000	2019-05-01
6	20	7000	2020-05-01
7	20	6000	2021-05-01

Idea: rank inside each department with `DENSE_RANK` (ties share a rank, no gaps) and keep rank 2. Use a subquery because a window function can't appear in `WHERE`.

```
SELECT dept_id, id, salary
FROM (SELECT dept_id, id, salary,
            DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
     FROM staff)
WHERE rnk = 2
ORDER BY dept_id, id;
```

Result

dept_id	id	salary
10	2	9000
10	3	9000
20	7	6000

Department 10 has 10000, 9000, 9000, 8000 — the second-highest *value* is 9000, held by **two** people, and both are returned. `ROW_NUMBER` would return only one; `RANK` would return the 8000 row (rank 4) instead.

4. Third-highest distinct salary *without* window functions

Idea: "the 3rd highest salary is the one where exactly **2 distinct salaries are higher**."

```
SELECT DISTINCT salary
FROM   emp e1
WHERE  2 = (SELECT COUNT(DISTINCT salary)
           FROM   emp e2
           WHERE  e2.salary > e1.salary);
```

Result

salary
65000

Distinct salaries in descending order are 90000, 70000, **65000**, ... so the third is 65000. Change `2` to `N - 1` for any `N`. This is the classic answer when the interviewer forbids `LIMIT` / `FETCH` / window functions.

5. Median salary

```
SELECT MEDIAN(salary) AS median_salary
FROM   emp;
```

Result

median_salary
47500

`MEDIAN` exists in **Oracle** (and PostgreSQL uses `PERCENTILE_CONT(0.5)`); MySQL and SQL Server have no built-in one, so know the portable method:

Idea: number the rows in salary order; the median is the middle row (odd count) or the average of the two middle rows (even count).

```
SELECT AVG(salary) AS median_salary
FROM   (SELECT salary,
              ROW_NUMBER() OVER (ORDER BY salary) AS rn,
              COUNT(*)   OVER ()      AS cnt
        FROM   emp)
WHERE  rn IN (FLOOR((cnt + 1) / 2), CEIL((cnt + 1) / 2));
```

Result

median_salary
47500

Ten salaries; the 5th and 6th smallest are 45000 and 50000 — median 47500. For an odd count both `FLOOR` and `CEIL` give the same middle row, so `AVG` returns it unchanged.

Part C — Gaps and islands

The most famous "hard" SQL pattern: finding **runs of consecutive values**.

Table `logins(user_id, login_day)` — one row per user per day they logged in.

logins

user_id	login_day
1	2025-08-01
1	2025-08-02
1	2025-08-03
1	2025-08-05
1	2025-08-06
1	2025-08-10
2	2025-08-01
2	2025-08-03
2	2025-08-04
2	2025-08-05
2	2025-08-06
2	2025-08-07

6. Consecutive-day login streaks

Idea (memorise this trick). Number each user's days `1, 2, 3, ...` with `ROW_NUMBER`. **Subtract that number from the date.** Within an unbroken run of consecutive days, the date goes up by 1 *and* the row number goes up by 1, so the **difference stays constant**; whenever a day is skipped the difference jumps. That constant is a label for the streak — group by it.

First, see the labels for user 1:

```
SELECT login_day,
       ROW_NUMBER() OVER (ORDER BY login_day) AS rn,
       login_day - ROW_NUMBER() OVER (ORDER BY login_day) AS grp
FROM   logins
WHERE  user_id = 1
ORDER BY login_day;
```

Result

login_day	rn	grp
2025-08-01	1	2025-07-31
2025-08-02	2	2025-07-31
2025-08-03	3	2025-07-31
2025-08-05	4	2025-08-01
2025-08-06	5	2025-08-01
2025-08-10	6	2025-08-04

Look at `grp` : 1–3 Aug all give `2025-07-31` ; 5–6 Aug both give `2025-08-01` ; 10 Aug gives `2025-08-04` . Three streaks, three labels. Now group by it:

```
SELECT user_id,
       MIN(login_day) AS streak_start,
       MAX(login_day) AS streak_end,
       COUNT(*)       AS days
FROM   (SELECT user_id, login_day,
              login_day - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_day) AS grp
        FROM   logins)
GROUP BY user_id, grp
ORDER BY user_id, streak_start;
```

Result

user_id	streak_start	streak_end	days
1	2025-08-01	2025-08-03	3
1	2025-08-05	2025-08-06	2
1	2025-08-10	2025-08-10	1
2	2025-08-01	2025-08-01	1
2	2025-08-03	2025-08-07	5

If the column were an integer (say `ticket_no`) the trick is simply `ticket_no - ROW_NUMBER() OVER (ORDER BY ticket_no)` . For dates, Oracle allows `date - number` ; in other databases use `DATEADD / date - INTERVAL` .

7. The longest streak per user, and users with a streak of 3 or more

```
SELECT user_id, MAX(days) AS longest_streak
FROM (SELECT user_id, COUNT(*) AS days
      FROM (SELECT user_id, login_day,
                    login_day - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_day) AS grp
              FROM logins)
      GROUP BY user_id, grp)
GROUP BY user_id
ORDER BY user_id;
```

Result

user_id	longest_streak
1	3
2	5

User 1's best run is 3 days (1–3 Aug); user 2's is 5 days (3–7 Aug). "Users with a streak of at least 3" is the same query with `HAVING COUNT(*) >= 3` on the inner grouping.

8. Missing numbers in a sequence

Table `tickets` — numbers should run 1 ... 10 but some were never issued.

tickets

ticket_no	issued_to
1	A
2	B
3	C
6	D
7	E
9	F
10	G

Idea: use `LEAD` to look at the **next** ticket number; where the difference is more than 1, a gap lies between.

```
SELECT ticket_no + 1 AS gap_from,
       next_no - 1 AS gap_to
FROM (SELECT ticket_no,
              LEAD(ticket_no) OVER (ORDER BY ticket_no) AS next_no
      FROM tickets)
WHERE next_no - ticket_no > 1
ORDER BY gap_from;
```

Result

gap_from	gap_to
4	5
8	8

Numbers 4–5 and 8 are missing. (The recursive-CTE number generator from Extra 1, `LEFT JOIN` ed to the table, also finds them and lists each missing number individually.)

Part D — Running totals, moving averages, growth

Table `monthly_sales(month, amount)`.

monthly_sales

month	amount
2025-01	100
2025-02	120
2025-03	90
2025-04	150
2025-05	150
2025-06	210

9. Running total and 3-month moving average

Idea: a window function with an `ORDER BY` accumulates; an explicit **frame** controls how many rows are included.

```
SELECT month, amount,
       SUM(amount) OVER (ORDER BY month ROWS UNBOUNDED PRECEDING)           AS running_total,
       ROUND(AVG(amount) OVER (ORDER BY month
                               ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1) AS moving_avg_3m
FROM   monthly_sales
ORDER  BY month;
```

Result

month	amount	running_total	moving_avg_3m
2025-01	100	100	100
2025-02	120	220	110
2025-03	90	310	103.3
2025-04	150	460	120
2025-05	150	610	130
2025-06	210	820	170

`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` = this month and the two before. The first two months average over fewer rows (e.g. February = $(100+120)/2 = 110$). Always write the frame explicitly — the default (`RANGE ... CURRENT ROW`) treats equal `ORDER BY` values as one, which surprises people.

10. Month-over-month growth

Idea: `LAG(amount)` fetches the previous row's value.

```
SELECT month, amount,
       LAG(amount) OVER (ORDER BY month) AS prev_amount,
       ROUND(100 * (amount - LAG(amount) OVER (ORDER BY month))
            / LAG(amount) OVER (ORDER BY month), 1) AS growth_pct
FROM   monthly_sales
ORDER BY month;
```

Result

month	amount	prev_amount	growth_pct
2025-01	100	NULL	NULL
2025-02	120	100	20
2025-03	90	120	-25
2025-04	150	90	66.7
2025-05	150	150	0
2025-06	210	150	40

January has no previous month, so `prev_amount` and `growth_pct` are `NULL` (and any arithmetic on `NULL` stays `NULL`). `LAG(amount, 1, 0)` would supply a default instead — but a zero would then make the division fail, so leaving `NULL` is right here.

Note on the SQLite check. The `100.0` in the executed copy only forces decimal division in the test engine; in Oracle `100 * ... / ...` is already decimal.

11. Best month and how it compares to the average

```
SELECT month, amount,
       ROUND(AVG(amount) OVER (), 1)           AS overall_avg,
       amount - ROUND(AVG(amount) OVER (), 1)   AS diff_from_avg
FROM   monthly_sales
ORDER BY amount DESC
FETCH FIRST 2 ROWS ONLY;
```

Result

month	amount	overall_avg	diff_from_avg
2025-06	210	136.7	73.3
2025-04	150	136.7	13.3

Part E — Pivot and unpivot

Table `qsales(product, quarter, amount)` — one row per product per quarter; Tablet had sales only in Q1 and Q3.

qsales

product	quarter	amount
Laptop	Q1	100
Laptop	Q2	150
Laptop	Q3	120
Laptop	Q4	200
Phone	Q1	300
Phone	Q2	250
Phone	Q3	400
Phone	Q4	350
Tablet	Q1	50
Tablet	Q3	80

12. Pivot: one row per product, one column per quarter

Idea (works everywhere): conditional aggregation — `SUM(CASE WHEN quarter = 'Q1' THEN amount END)`.

```
SELECT product,
       SUM(CASE WHEN quarter = 'Q1' THEN amount END) AS q1,
       SUM(CASE WHEN quarter = 'Q2' THEN amount END) AS q2,
       SUM(CASE WHEN quarter = 'Q3' THEN amount END) AS q3,
       SUM(CASE WHEN quarter = 'Q4' THEN amount END) AS q4,
       SUM(amount)                                     AS total
FROM   qsales
GROUP BY product
ORDER BY product;
```

Result

product	q1	q2	q3	q4	total
Laptop	100	150	120	200	570
Phone	300	250	400	350	1300
Tablet	50	NULL	80	NULL	130

Tablet's missing quarters come out as `NULL` (no `ELSE` → the `CASE` returns `NULL` → `SUM` ignores it). Wrap with `NVL(..., 0)` if you want zeros.

Oracle 11g+ also has a dedicated `PIVOT` clause, the same result with less typing:

```
SELECT * FROM (SELECT product, quarter, amount FROM qsales)
PIVOT (SUM(amount) FOR quarter IN ('Q1' AS q1, 'Q2' AS q2, 'Q3' AS q3, 'Q4' AS q4));
```

13. Unpivot: columns back into rows

Table `wide_sales(product, q1, q2, q3, q4)`.

wide_sales

product	q1	q2	q3	q4
Laptop	100	150	120	200
Phone	300	250	400	350
Tablet	50	NULL	80	NULL

Idea (works everywhere): one `SELECT` per column, glued with `UNION ALL`.

```
SELECT product, 'Q1' AS quarter, q1 AS amount FROM wide_sales WHERE q1 IS NOT NULL
UNION ALL
SELECT product, 'Q2', q2 FROM wide_sales WHERE q2 IS NOT NULL
UNION ALL
SELECT product, 'Q3', q3 FROM wide_sales WHERE q3 IS NOT NULL
UNION ALL
SELECT product, 'Q4', q4 FROM wide_sales WHERE q4 IS NOT NULL
ORDER BY product, quarter;
```

Result

product	quarter	amount
Laptop	Q1	100
Laptop	Q2	150
Laptop	Q3	120
Laptop	Q4	200
Phone	Q1	300
Phone	Q2	250
Phone	Q3	400
Phone	Q4	350
Tablet	Q1	50
Tablet	Q3	80

(Oracle's `UNPIVOT (amount FOR quarter IN (q1, q2, q3, q4))` does it in one clause and, by default, also drops the NULLs.)

Part F — Percentages and top-N

14. Cumulative percentage (Pareto: "which products make up 80% of sales?")

```
SELECT product, total,
       ROUND(100 * SUM(total) OVER (ORDER BY total DESC ROWS UNBOUNDED PRECEDING)
           / SUM(total) OVER (), 1) AS cumulative_pct
FROM   (SELECT product, SUM(amount) AS total FROM qsales GROUP BY product)
ORDER BY total DESC;
```

Result

product	total	cumulative_pct
Phone	1300	65
Laptop	570	93.5
Tablet	130	100

Phone alone is 65% of sales; Phone and Laptop together are 93.5%. **Nested aggregation:** the inner query does the `GROUP BY`, the outer query applies window functions over the *grouped* result.

15. Top 2 products in each quarter

```
SELECT quarter, product, amount
FROM   (SELECT quarter, product, amount,
              DENSE_RANK() OVER (PARTITION BY quarter ORDER BY amount DESC) AS rnk
        FROM   qsales)
WHERE  rnk <= 2
ORDER BY quarter, amount DESC;
```

Result

quarter	product	amount
Q1	Phone	300
Q1	Laptop	100
Q2	Phone	250
Q2	Laptop	150
Q3	Phone	400
Q3	Laptop	120
Q4	Phone	350
Q4	Laptop	200

Q2 and Q4 have only two rows each so both appear; Q1 and Q3 have three, and the smallest (Tablet) is cut. Because `DENSE_RANK` is used, if two products *tied* for second place both would be returned.

Part G — Relational division and updates

16. Customers who bought **every** product

Tables `products` and `purchases` (Chitra bought the Bag twice).

products

product_id	name
1	Pen
2	Book
3	Bag

purchases

customer	product_id
Asha	1
Asha	2
Asha	3
Bala	1
Bala	2
Chitra	1
Chitra	2
Chitra	3
Chitra	3

Idea ("relational division"): count the *distinct* products each customer bought and compare with the total number of products.

```
SELECT customer
FROM purchases
GROUP BY customer
HAVING COUNT(DISTINCT product_id) = (SELECT COUNT(*) FROM products)
ORDER BY customer;
```

Result

customer
Asha
Chitra

Bala bought only 2 of 3 products, so is excluded. **DISTINCT is essential:** without it Chitra's 4 rows (Bag twice) would give the wrong count of 4, not 3.

17. Swap two values with one UPDATE

Table `members` — someone entered `F` and `M` the wrong way round.

Idea: a single `UPDATE` with `CASE` evaluates every row against the **old** values, so both changes happen "at once". Two separate `UPDATE`s would turn everyone into the same value.

```
UPDATE members
SET    gender = CASE gender WHEN 'F' THEN 'M'
                        WHEN 'M' THEN 'F'
END;
```

Table after

id	name	gender
1	Asha	M
2	Bala	F
3	Chitra	M
4	Dinesh	F

Part H — Self-join puzzles on `emp`

18. Employees who earn more than their manager

The sample data has none — a good result to explain, not a bug.

```
SELECT e.emp_name AS employee, e.salary AS emp_salary,
       m.emp_name AS manager,  m.salary AS mgr_salary
FROM   emp e
JOIN   emp m ON e.manager_id = m.emp_id
WHERE  e.salary > m.salary;
```

Result

employee	emp_salary	manager	mgr_salary
(no rows)			

Joining `emp` to itself (one copy as the employee, one as the manager) is the pattern. The CEO (no manager) drops out of an inner join; use `LEFT JOIN` to keep them.

19. Pairs of employees with the same salary

Idea: self join on equal salary, and use `a.emp_id < b.emp_id` so each pair appears once and nobody pairs with themself.

```
SELECT a.emp_name AS employee_1, b.emp_name AS employee_2, a.salary
FROM   emp a
JOIN   emp b ON a.salary = b.salary AND a.emp_id < b.emp_id
ORDER BY a.salary DESC;
```

Result

employee_1	employee_2	salary
Chitra	Dinesh	50000
Gita	Hari	35000

20. Managers with at least two direct reports

```
SELECT m.emp_name AS manager, COUNT(*) AS direct_reports
FROM   emp e
JOIN   emp m ON e.manager_id = m.emp_id
GROUP BY m.emp_id, m.emp_name
HAVING COUNT(*) >= 2
ORDER BY direct_reports DESC, manager;
```

Result

manager	direct_reports
Asha	3
Bala	3
Farhan	2

Group by the manager's **id** as well as the name — two managers could share a name.

Part I — More window-function patterns

21. Rows into a comma-separated list

```
SELECT dept_id,
       LISTAGG(emp_name, ', ' ) WITHIN GROUP (ORDER BY emp_name) AS employees
FROM   emp
WHERE  dept_id IS NOT NULL
GROUP BY dept_id
ORDER BY dept_id;
```

Result

dept_id	employees
10	Asha

dept_id	employees
20	Bala, Chitra, Dinesh, Esha, Isha
30	Farhan, Gita, Hari

`LISTAGG` is Oracle (11gR2+). Others: MySQL `GROUP_CONCAT`, PostgreSQL `STRING_AGG`, SQL Server `STRING_AGG` (2017+). Oracle raises **ORA-01489** if the result exceeds 4000 bytes (later versions add `ON OVERFLOW TRUNCATE`).

22. First and last value in a group — and the frame trap

"Next to every employee show the highest-paid and lowest-paid person in their department."

```
SELECT dept_id, emp_name, salary,
       FIRST_VALUE(emp_name) OVER (PARTITION BY dept_id ORDER BY salary DESC, emp_name) AS
top_earner,
       LAST_VALUE(emp_name) OVER (PARTITION BY dept_id ORDER BY salary DESC, emp_name
                                ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
lowest_earner
FROM   emp
WHERE  dept_id IN (20, 30)
ORDER BY dept_id, salary DESC, emp_name;
```

Result

dept_id	emp_name	salary	top_earner	lowest_earner
20	Bala	70000	Bala	Isha
20	Chitra	50000	Bala	Isha
20	Dinesh	50000	Bala	Isha
20	Esha	40000	Bala	Isha
20	Isha	20000	Bala	Isha
30	Farhan	65000	Farhan	Hari
30	Gita	35000	Farhan	Hari
30	Hari	35000	Farhan	Hari

The frame trap. With an `ORDER BY`, the default frame ends at the **current row**, so `LAST_VALUE` would return the *current* row's value, not the last row of the group. Extending the frame to `UNBOUNDED FOLLOWING` (as above) fixes it. `FIRST_VALUE` doesn't have this problem because the default frame always starts at the beginning.

23. The three most recently hired

```
SELECT emp_name, hire_date
FROM   emp
ORDER BY hire_date DESC
FETCH  FIRST 3 ROWS ONLY;
```

Result

emp_name	hire_date
Isha	2024-01-08
Jai	2023-04-01
Hari	2022-06-18

24. Compare each row with the previous one

"Who was paid more than the employee hired just before them?"

```
SELECT emp_name, hire_date, salary, prev_salary
FROM (SELECT emp_name, hire_date, salary,
             LAG(salary) OVER (ORDER BY hire_date) AS prev_salary
      FROM emp)
WHERE salary > prev_salary
ORDER BY hire_date;
```

Result

emp_name	hire_date	salary	prev_salary
Jai	2023-04-01	45000	35000

The first hire has `prev_salary = NULL`, and `salary > NULL` is UNKNOWN, so that row is filtered out — no special handling needed.

25. Split employees into three salary bands

```
SELECT emp_name, salary,
       NTILE(3) OVER (ORDER BY salary DESC, emp_id) AS band
FROM emp
ORDER BY salary DESC, emp_id;
```

Result

emp_name	salary	band
Asha	90000	1
Bala	70000	1
Farhan	65000	1
Chitra	50000	1
Dinesh	50000	2
Jai	45000	2
Esha	40000	2
Gita	35000	3
Hari	35000	3
Isha	20000	3

`NTILE(3)` deals the 10 rows into 3 buckets as evenly as possible; the leftover row goes to the *first* bucket, so the bucket sizes are 4, 3, 3.

Part J — Groups, NULL traps and aggregates of aggregates

26. Departments where **every** employee earns more than 30,000

Idea: "every row satisfies X" is the same as "the **minimum** satisfies X".

```
SELECT d.dept_name, MIN(e.salary) AS lowest_salary
FROM   dept d
JOIN   emp e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING MIN(e.salary) > 30000
ORDER BY d.dept_name;
```

Result

dept_name	lowest_salary
Management	90000
Sales	35000

Engineering fails because Isha earns 20000. HR doesn't appear either: with no employees it has no rows to join (use an outer join and decide what "every" means for an empty group).

27. Employees who report to a manager in a **different** department — the NULL trap

```
SELECT e.emp_name, e.dept_id AS emp_dept, m.emp_name AS manager, m.dept_id AS mgr_dept
FROM   emp e
JOIN   emp m ON e.manager_id = m.emp_id
WHERE  e.dept_id <> m.dept_id
ORDER BY e.emp_name;
```

Result

emp_name	emp_dept	manager	mgr_dept
Bala	20	Asha	10
Farhan	30	Asha	10

Jai is missing. Jai reports to Asha but has no department (`NULL`), and `NULL <> 10` is UNKNOWN, so the row is rejected. If "different" should include "unknown", make the NULL explicit:

```

SELECT e.emp_name, e.dept_id AS emp_dept, m.emp_name AS manager, m.dept_id AS mgr_dept
FROM   emp e
JOIN   emp m ON e.manager_id = m.emp_id
WHERE  NVL(e.dept_id, -1) <> NVL(m.dept_id, -1)
ORDER BY e.emp_name;

```

Result

emp_name	emp_dept	manager	mgr_dept
Bala	20	Asha	10
Farhan	30	Asha	10
Jai	NULL	Asha	10

(PostgreSQL has `IS DISTINCT FROM` for this; Oracle doesn't — `NVL / COALESCE` or `DECODE(a, b, 0, 1) = 1` are the Oracle idioms.)

28. Hires per year with a cumulative headcount

Idea: a window function can wrap an **aggregate** — `SUM(COUNT(*)) OVER (ORDER BY ...)` sums the per-year counts as they accumulate.

```

SELECT EXTRACT(YEAR FROM hire_date)                AS hire_year,
       COUNT(*)                                     AS hired,
       SUM(COUNT(*)) OVER (ORDER BY EXTRACT(YEAR FROM hire_date)) AS headcount_so_far
FROM   emp
GROUP BY EXTRACT(YEAR FROM hire_date)
ORDER BY hire_year;

```

Result

hire_year	hired	headcount_so_far
2015	1	1
2016	1	2
2017	1	3
2018	1	4
2019	1	5
2020	1	6
2021	1	7
2022	1	8
2023	1	9
2024	1	10

The `GROUP BY` runs first, producing one row per year; the window function then runs over those grouped rows (this is the execution order from Chapter 04).

Patterns to recognise

The question sounds like...	Reach for
"consecutive ...", "streak", "in a row"	gaps and islands: <code>value - ROW_NUMBER()</code> , then <code>GROUP BY</code>
"missing / gap in a sequence"	<code>LEAD / LAG</code> and compare, or a generated series + anti-join
"top N per group", "N-th highest"	<code>DENSE_RANK / ROW_NUMBER OVER (PARTITION BY ...)</code> in a subquery
"running total", "moving average"	<code>SUM/AVG ... OVER (ORDER BY ... ROWS ...)</code>
"compared with the previous / next"	<code>LAG / LEAD</code>
"rows to columns" / "columns to rows"	conditional aggregation (<code>PIVOT</code>) / <code>UNION ALL (UNPIVOT)</code>
"customers who did all of ..."	<code>HAVING COUNT(DISTINCT x) = (total)</code>
"customers who did none of ..."	<code>NOT EXISTS / LEFT JOIN ... IS NULL</code>
"duplicates"	<code>GROUP BY ... HAVING COUNT(*) > 1</code> , delete with <code>MIN(id)</code> or <code>ROW_NUMBER</code>
"every row satisfies ..."	<code>HAVING MIN(x) ... / MAX(x) ...</code> , or <code>NOT EXISTS</code> of a violator
"pairs", "same X as another row"	self join with <code>a.id < b.id</code>
"hierarchy of any depth"	recursive CTE / <code>CONNECT BY</code> (Extra 1)

Interview corner

Q. How do you find and delete duplicate rows? Group by the columns that define a duplicate with `HAVING COUNT(*) > 1` to find them; delete all but the `MIN(id)` per group, or number rows with `ROW_NUMBER() OVER (PARTITION BY ...)` and delete numbers above 1 (in Oracle you can also compare `ROWID`). Prevent it with a unique constraint.

Q. Explain how to find consecutive-day streaks. Assign `ROW_NUMBER` per user ordered by date, subtract it from the date; within a streak the result is constant, so `GROUP BY` that value and take `MIN` , `MAX` and `COUNT` .

Q. RANK vs DENSE_RANK for "N-th highest"? `DENSE_RANK` — it doesn't skip numbers after ties, so the second-highest *value* is always rank 2.

Q. How do you compute a median without `MEDIAN` ? Number the rows with `ROW_NUMBER` , count them, and average the row(s) at positions `FLOOR((n+1)/2)` and `CEIL((n+1)/2)` ; or `PERCENTILE_CONT(0.5)` .

Q. Pivot without `PIVOT` ? `SUM(CASE WHEN col = 'x' THEN val END)` per output column with a `GROUP BY` on the row key.

Q. Find customers who bought all products. Group by customer and require `COUNT(DISTINCT product_id)` equals the total number of products.

Q. Why does `LAST_VALUE` often give the wrong answer? The default frame stops at the current row; specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` .

Q. What happens to `NULL` s in `LAG` , `<>` and `NOT IN` ? `LAG` returns `NULL` for the first row; `NULL <> x` is `UNKNOWN` so the row is filtered out; `NOT IN` with a `NULL` in the list returns nothing. In all three, decide deliberately how `NULL` should be treated.

Quick recap

- **Duplicates:** `GROUP BY ... HAVING COUNT(*) > 1 ; delete NOT IN (SELECT MIN(id) ...)` or `ROW_NUMBER > 1 .`
- **N-th highest:** `DENSE_RANK ... = N`, or "N-1 distinct values are higher".
- **Gaps and islands:** `value - ROW_NUMBER()` = streak label. **Gaps:** `LEAD` and compare.
- **Running / moving:** window frame `ROWS BETWEEN ... PRECEDING AND CURRENT ROW ; growth: LAG .`
- **Pivot** = `SUM(CASE ...)` ; **unpivot** = `UNION ALL ; division` = `HAVING COUNT(DISTINCT) = total .`
- **Frame trap** for `LAST_VALUE` ; **NULL traps** in `<>`, `NOT IN`, `LAG` .